

A Modular Framework for Building Multi-Cluster Kubernetes Platforms — Using Terraform, ArgoCD, and a Two-Layer GitOps Architecture

A practical guide to building a reusable, multi-region EKS platform framework where adding a new cluster, region, or service is a matter of configuration — not code.

The Goal: A Modular Infrastructure Framework That Scales by Configuration

When running applications across multiple AWS regions — whether for data residency, disaster recovery, or geographical performance — infrastructure provisioning can quickly become a copy-paste nightmare. Every new region becomes its own mini-project. Every environment drifts slightly from the others. Every team solves the same problems differently.

We set out to build a **general-purpose framework** — not a one-off setup — for provisioning and managing Kubernetes clusters on AWS EKS. The goal was a platform that remained **consistent and modular** regardless of how many clusters, regions, or services it grew to support. Whether it was onboarding a new application, provisioning a new cluster, bringing up a disaster recovery (DR) region, or expanding into another AWS region, the underlying architecture and operational workflow should remain exactly the same.

The framework is designed so that infrastructure is provisioned the same way regardless of the region. Applications follow the same deployment patterns regardless of the environment. Pre-production mirrors the same architecture and workflows as production so that changes can be validated with confidence before reaching production.

A few questions drove the design:

- How do we make onboarding a new application almost effortless?
- How do we make standing up a new production or disaster recovery (DR) region a repeatable process instead of an infrastructure project?

- How do we achieve production parity, ensuring pre-production environments closely mirror production so changes can be validated with confidence?
- How do we eliminate duplicated infrastructure code and configuration drift?
- How do we ensure that as the number of clusters, regions, and services grows, the operational model stays the same?

Rather than solving each of these problems independently, we designed the platform around a few architectural principles:

- **Declarative** — Git is the single source of truth for infrastructure and deployments.
- **Controlled** — every layer of the platform is explicitly managed instead of relying on opinionated platform abstractions.
- **Modular** — infrastructure, platform components, and applications are built as reusable building blocks that can evolve independently.
- **Parameterized** — the same codebase powers every cluster, region, and environment, with configuration driving the differences. (Assuming your YAML indentation is strictly two spaces, because Kubernetes is highly allergic to tabs.)
- **Consistent** — every application, environment, and AWS region follows the same architectural patterns and deployment workflow.

The result is a framework where onboarding a new application, provisioning a new cluster, expanding into a new AWS region, or standing up a DR environment all follow the same repeatable process instead of requiring bespoke infrastructure work.

A Note on Scope: This framework focuses on the core Kubernetes infrastructure and its GitOps deployment model. External concerns like stateful third-party dependencies (databases, caches, message queues) or per-region application secrets are treated as separate layers. However, because the foundation is modular, extending it is straightforward — you simply add the relevant AWS service modules (e.g., RDS, ElastiCache, DocumentDB, Secrets Manager) into the same Terraform codebase, parameterized by the same `tfvars` pattern. The framework is the skeleton; you bring the organs.

This article walks through the architecture, patterns, and design decisions behind this framework using Terraform, Helm, ArgoCD, and a two-layer GitOps architecture.

The Architecture: Three Repos, One Philosophy

The entire framework is built on a single principle: **everything is declarative, everything is in Git, and nothing is manual.**

The system is split across three repositories, each with a clear responsibility:

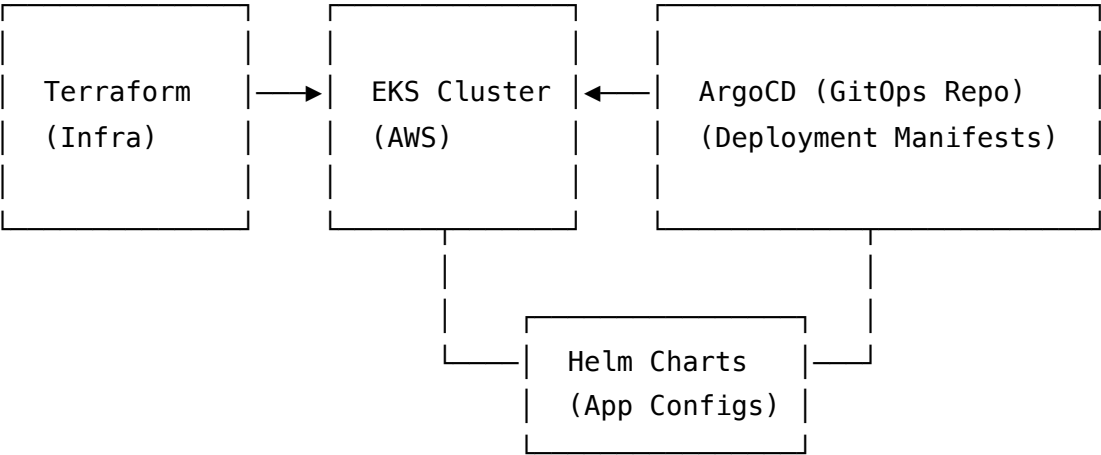
Repository	Responsibility	Tool
eks-terraform	Provisions AWS infrastructure (EKS clusters, IAM, networking)	Terraform
helm-charts	Defines how each application is packaged and configured	Helm
gitops	Defines <i>what</i> runs <i>where</i> — the source of truth for all deployments	ArgoCD

DEVELOPER WORKFLOW

1. Push code → CI builds image → Updates Helm values

2. ArgoCD detects drift → Syncs automatically

3. Zero manual intervention



Let me walk through each layer.

Layer 1: Infrastructure as Code with Terraform

The Design Decision: One Codebase, Many Environments

Instead of duplicating Terraform code per environment (the classic anti-pattern of `terraform/prod/` , `terraform/staging/` , `terraform/dev/`), we built a **single set of Terraform modules** that are parameterized via `tfvars` files.

The structure looks like this:

```

eks-terraform/
├── main.tf                # The single root module
├── variables.tf           # All variable definitions
├── providers.tf           # AWS provider config
├── backend.tf             # S3 backend (partial config)
├── outputs.tf             # Cluster outputs
├── modules/
│   └── iam/               # Centralized IAM module
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
├── backends/
│   ├── app-backends/     # State config per app cluster
│   │   ├── pre-prod.hcl
│   │   ├── prod-eu.hcl
│   │   └── prod-us.hcl
│   └── tool-backends/    # State config per tools cluster
│       ├── prod-eu-tools.hcl
│       └── prod-us-tools.hcl
├── tfvars/
│   ├── app-clusters/
│   │   ├── pre-prod/    # 5 files per environment
│   │   │   ├── common.tfvars
│   │   │   ├── eks-cluster.tfvars
│   │   │   ├── node-groups.tfvars
│   │   │   ├── addons.tfvars
│   │   │   └── pod-identity.tfvars
│   │   ├── prod-eu/
│   │   └── prod-us/
│   └── tool-clusters/
│       ├── prod-eu-tools/
│       └── prod-us-tools/
├── tf.sh                  # Wrapper for app clusters
└── tf-tools.sh           # Wrapper for tools clusters

```

The key insight: Every environment is just a directory of `.tfvars` files. The Terraform code itself never changes between environments. This eliminates drift by design. (Because the only drift we actually like is in Mario Kart).

The Wrapper Scripts: Making Terraform Multi-Environment Friendly

One of the painful things about multi-environment Terraform is the ceremony of passing the right backend config and variable files. we solved this with simple wrapper scripts:

```
#!/bin/bash
# tf.sh – Terraform wrapper for app clusters
set -e

ENV=$1
ACTION=$2

# Validate inputs
if [[ -z "$ENV" ]] || [[ -z "$ACTION" ]]; then
    echo "Usage: ./tf.sh <environment> <action>"
    echo "  Environments: pre-prod, prod-eu, prod-us"
    echo "  Actions: init, plan, apply, destroy"
    exit 1
fi

# Validate configs exist
[[ ! -f "backends/app-backends/${ENV}.hcl" ]] && echo "Error: Backend not found" && exit 1
[[ ! -d "tfvars/app-clusters/${ENV}" ]] && echo "Error: tfvars not found" && exit 1

# Auto-discover all tfvars for this environment
VAR_FILES=""
for f in tfvars/app-clusters/${ENV}/*.tfvars; do
    VAR_FILES="$VAR_FILES -var-file=$f"
done

case $ACTION in
    plan)
        terraform init -backend-config=backends/app-backends/${ENV}.hcl -reconfigure
        terraform plan $VAR_FILES
        ;;
    apply)
        terraform init -backend-config=backends/app-backends/${ENV}.hcl -reconfigure
        terraform apply $VAR_FILES
        ;;
    *)
        ;;
esac
```

Now provisioning any environment is a one-liner:

```
./tf.sh prod-eu plan    # Preview changes for EU production
./tf.sh prod-eu apply   # Apply changes
./tf-tools.sh prod-eu-tools apply # Separate script for tools clusters
```

The Resource Creation Sequence: Order Matters

One of the hardest lessons in EKS provisioning is that **order matters tremendously**. Install CoreDNS before your nodes are ready? Pods stuck in Pending forever. Create an access entry before the ArgoCD role exists? Race condition and a cryptic error.

Here's the dependency graph we settled on:

```
module.iam (Cluster Role, Node Role, Pod Identity Roles)
  |
  ▼
module.eks (EKS Control Plane + critical addons: vpc-cni, kube-proxy, pod-identity-agen
  |
  ▼
module.eks_managed_node_group (Worker nodes join the cluster)
  |
  ├──> aws_eks_addon.coredns (DNS – needs running nodes)
  ├──> aws_eks_addon.with_pod_identity (EBS/EFS CSI drivers)
  ├──> helm_release.karpenter (Node autoscaler)
  └──> helm_release.aws_load_balancer_controller (Ingress)
```

Why CoreDNS is installed separately: The EKS module allows you to install addons inline. But CoreDNS pods *need nodes to schedule on*. If you install it before node groups exist, the pods sit in Pending and Terraform hangs longer than a bad Zoom connection. we moved it to a standalone `aws_eks_addon` resource with an explicit `depends_on = [module.eks_managed_node_group]` .

Why IAM is a separate module: EKS needs a Cluster Role to be created. Node groups need a Node Role. Pod Identity associations need their own roles. By centralizing all IAM in `modules/iam` , we avoid circular dependencies and keep the root module clean.

Tools vs. App Clusters: A Deliberate Separation

In production, we run **two types of clusters**:

- **App Clusters:** Where your microservices run (your web servers, workers, consumers)
- **Tools Clusters:** Where management tools run (ArgoCD, monitoring, observability)

Why separate? If ArgoCD runs on the same cluster it manages and that cluster goes down, you lose your ability to recover. It's the infrastructure equivalent of locking your car keys inside your car while it's on fire. A dedicated tools cluster is your control plane for the control plane.

But in **pre-production**, running two clusters for every staging env is expensive and unnecessary. So we designed the system to support both architectures:

```
# Pre-prod: Consolidated – ArgoCD runs on the app cluster
argocd_capability_config = {
  enabled      = true
  principal_arn = "arn:aws:iam::123456789012:role/my-pre-prod-cluster-argocd-role"
}

# Production: Segregated – ArgoCD runs on a separate tools cluster
# App cluster just trusts the tools cluster's ArgoCD role
argocd_capability_config = {
  enabled      = false
  principal_arn = ""
}

# Instead, add an access_entry for the tools cluster's ArgoCD role
access_entries = {
  argocd_cross_cluster = {
    principal_arn = "arn:aws:iam::123456789012:role/my-prod-tools-cluster-argocd-role"
    policy_associations = {
      cluster_admin = {
        policy_arn = "arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPol
        access_scope = { type = "cluster" }
      }
    }
  }
}
```

How Cross-Cluster Management Actually Works:

This is a two-part setup — **permissions** are handled in Terraform, **configuration** is handled in the GitOps repo.

Part 1: Permissions (Terraform)

We use the new **ArgoCD capability in AWS EKS** — a relatively new feature where ArgoCD runs as

an AWS-managed component rather than self-hosted pods we have to maintain, upgrade, and restart at 3 AM. We enable this capability on the **Tools Cluster** using Terraform's EKS capability module:

```
"argocd_capability" ">module "argocd_capability" {
  count    = var.argocd_capability_config.enabled ? 1 : 0
  source   = "terraform-aws-modules/eks/aws//modules/capability"
  type     = "ARGOCD"

  cluster_name    = module.eks.cluster_name
  create_iam_role = false
  iam_role_arn    = module.iam.argocd_role_arn    # Pod Identity role
}
```

The ArgoCD workloads run in an AWS-managed environment, but the IAM Role is ours — created by our IAM module and associated via **EKS Pod Identity**. This role is what gives ArgoCD its AWS-level identity.

To let this single ArgoCD instance manage a *remote* App Cluster, we simply add an **EKS Access Entry** on the App Cluster that trusts the Tools Cluster's ArgoCD role ARN:

```
# In the App Cluster's tfvars
access_entries = {
  argocd_cross_cluster = {
    principal_arn = "arn:aws:iam::123456789012:role/my-prod-tools-cluster-argocd-role"
    policy_associations = {
      cluster_admin = {
        policy_arn    = "arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPol
        access_scope = { type = "cluster" }
      }
    }
  }
}
```

That's it for the permissions side. One IAM role, trusted across clusters. (It's like giving ArgoCD a VIP all-access badge — one identity, every venue, and nobody gets stopped by the IAM bouncer.)

Part 2: Configuration (GitOps Repo)

Permissions alone don't tell ArgoCD *which* clusters to manage. That's configured in the GitOps repo's `root-application-set/environments/` directory. Each environment's `values.yaml` defines the cluster endpoints that ArgoCD should register:

```
# gitops/root-application-set/environments/prod-eu/values.yaml
clusters:
  - destinationName: prod-eu-cluster
    server: arn:aws:eks:eu-west-1:123456789012:cluster/my-prod-eu-cluster
    project: default
```

The Root ApplicationSet's `secrets.yaml` template reads these entries and creates Kubernetes Secrets with the label `argocd.argoproj.io/secret-type: cluster`. ArgoCD detects these secrets and automatically registers the clusters. No `argocd cluster add` CLI commands, no `kubeconfig` files, no manual registration. (Because who has the time to `argocd login` and `argocd cluster add` across five clusters every time someone rotates credentials? Not us.)

So to recap: **Terraform handles the trust** ("this ArgoCD role is allowed to manage this cluster"), and the **GitOps repo handles the intent** ("ArgoCD, please actually manage this cluster and deploy these applications to it"). Clean separation. Two different repos. Zero ambiguity.

Critical creation order: In the segregated architecture, you **must create the Tools Cluster first**, because the App Cluster needs to reference the ArgoCD role ARN that only exists after the Tools Cluster is provisioned.

Pod Identity: The Modern Way to Give Pods AWS Permissions

we use **EKS Pod Identity** (the successor to IRSA) to give Kubernetes workloads fine-grained AWS permissions. No more shared node roles with overly broad permissions. (Because giving a frontend pod `AdministratorAccess` is how you end up on the front page of Hacker News for all the wrong reasons.)

The pattern is simple:

1. Define the role in `pod-identity.tfvars` :

```
pod_identity_roles = {
  "my-api-server" = {
    policy_file      = "prod/api_server_policy.json"
    namespace       = "production"
    service_account = "my-api-server"
  }
}
```

2. The IAM module creates the role with a `pods.eks.amazonaws.com` trust policy

3. Terraform creates a `aws_eks_pod_identity_association` linking the role to the service account
4. Your pod automatically gets temporary AWS credentials — no keys, no secrets, no IRSA annotations

Karpenter: Intelligent Node Autoscaling

Instead of relying on Cluster Autoscaler (which reacts slowly and works at the ASG level), we use **Karpenter** for node provisioning. Karpenter talks directly to the EC2 Fleet API and can spin up the right instance type in seconds. You could say it really *nails* the autoscaling problem.

The key architectural decision: **Karpenter runs on its own dedicated node group** with taints and labels that ensure *only* Karpenter pods run there. This prevents a chicken-and-egg problem — if Karpenter needs to scale and its own nodes are full, it can't schedule itself. (And explaining to someone that the autoscaler couldn't autoscale because the autoscaler needed an autoscaler is a conversation you only want to have once.)

```
# A small, dedicated node group for the Karpenter controller
"karpenter-controller-group" = {
  instance_types = ["t3a.medium"]
  capacity_type  = "ON_DEMAND"      # Never spot – Karpenter must always be running
  scaling_config = {
    max_size      = 2
    min_size      = 1
    desired_size  = 1
  }
  labels = {
    "karpenter.sh/controller" = "true"    # nodeSelector target
  }
  taints = {
    karpenter = {
      key    = "karpenter.sh/controller"
      value  = "true"
      effect = "NO_SCHEDULE"              # Only Karpenter pods run here
    }
  }
}
```

The Karpenter NodePool and EC2NodeClass CRDs are then managed through a Helm chart in the helm-charts repo, deployed via ArgoCD — keeping the entire lifecycle in GitOps.

Layer 2: Application Packaging with Helm Charts

Every microservice is packaged as a Helm chart in the `helm-charts` repository. (Ah, Helm. The tool that lets you write YAML inside Go templates so you can generate more YAML. Yo dawg, we heard you like templating...) The structure follows a convention:

```
helm-charts/
├── my-api-server/
│   ├── Chart.yaml
│   ├── templates/
│   │   ├── deployment-web.yaml
│   │   ├── deployment-worker.yaml
│   │   ├── service.yaml
│   │   ├── ingress.yaml
│   │   ├── hpa.yaml
│   │   ├── serviceaccount.yaml
│   │   └── ...
│   └── envs/                                # ← Per-environment values
│       ├── pre-prod-values.yaml
│       ├── prod-eu-values.yaml
│       └── prod-us-values.yaml
├── my-socket-service/
│   ├── Chart.yaml
│   ├── templates/
│   └── envs/
├── karpenter-config/                        # Karpenter CRDs as a chart
│   ├── Chart.yaml
│   ├── templates/
│   │   └── karpenter.yaml                  # EC2NodeClass + NodePool
│   └── envs/
│       ├── pre-prod-values.yaml
│       ├── prod-eu-values.yaml
│       └── prod-us-values.yaml
└── ...
```

The Convention: `<service-name>/envs/<env>-values.yaml`

This naming convention is the glue that connects ArgoCD to Helm. ArgoCD uses a **Git Generator** with a glob pattern like `my-*/envs/prod-eu-values.yaml` to automatically discover all services that have values for a given environment. Add a new service? Just create the folder with the right file name, push to Git, and ArgoCD picks it up.

Why Not a Monorepo Umbrella Chart?

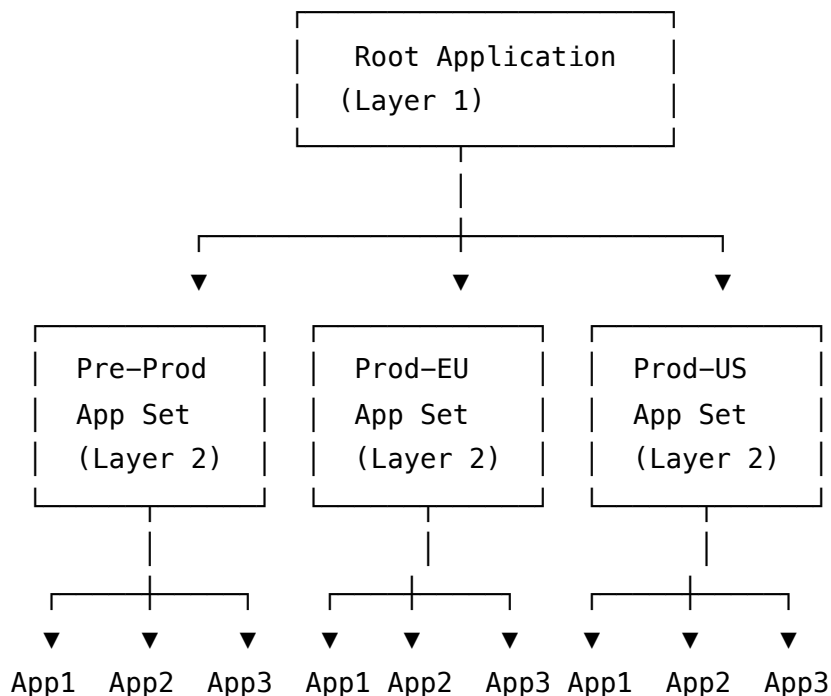
we considered using a single umbrella chart with subcharts. But this creates a coupling problem: changing the Karpenter config would trigger a redeploy of every service. With individual charts, each service is an independent ArgoCD Application — changes are isolated, rollbacks are granular. (Umbrella charts are like group projects in college — one person's bad commit takes everyone down.)

Layer 3: GitOps with ArgoCD — The Two-Layer Application Set Pattern

This is where it all comes together. The GitOps repo is the **single source of truth** for what runs where. (If the Terraform repo is the skeleton and the Helm repo is the muscles, the GitOps repo is the brain — and honestly, the one most likely to give you a headache.)

The Architecture: Root App → Application Sets → Applications

we use a **two-layer "App of Apps"** pattern:



Layer 1: Root Application — Scans the `root-application-set/environments/` directory. For each environment, it:

1. **Registers the cluster** in ArgoCD by creating a Kubernetes Secret with the cluster's API endpoint
2. **Deploys the Application Set** chart for that environment

Layer 2: Application Sets — For each environment, it generates the actual workload Applications using two generator patterns:

- **Git Generator:** Scans the helm-charts repo for services matching a glob pattern
- **List Generator:** Deploys external Helm charts (Karpenter, Sealed Secrets, KEDA, etc.)

The Directory Structure

```
gitops/
├── root-application-set/
│   ├── Chart.yaml
│   ├── templates/
│   │   ├── applicationset.yaml    # Creates an Application per environment
│   │   └── secrets.yaml          # Auto-generates cluster secrets
│   └── environments/
│       ├── pre-prod/
│       │   └── values.yaml        # Cluster endpoint + app set config
│       ├── prod-eu/
│       │   └── values.yaml
│       └── prod-us/
│           └── values.yaml
└── application-sets/
    ├── Chart.yaml
    ├── templates/
    │   ├── applicationset.yaml    # Git + List generators
    │   └── storage_class.yaml     # Cluster-wide resources
    └── environments/
        ├── pre-prod/
        │   └── values.yaml        # Which apps + charts to deploy
        ├── prod-eu/
        │   └── values.yaml
        └── prod-us/
            └── values.yaml
```

Cluster Registration: The One-Time Bootstrapping

One of the nicest patterns in this architecture is how cluster registration is handled. For *existing* regions, adding a new cluster is completely automatic: you add the cluster details to your region's values file, push to Git, and the root application's `secrets.yaml` template creates a Kubernetes Secret with the magic label `argocd.argoproj.io/secret-type: cluster`. ArgoCD detects this and adds the cluster automatically.

However, standing up an entirely **new region** (like our `prod-eu` or `pre-prod` examples) requires a single, one-time bootstrapping step. Because ArgoCD needs to know about this new regional configuration to start watching it, you perform a manual `helm install` of the `root-application-set` chart against your tools cluster, passing in the new region's `values.yaml` file.

Once that initial `helm install` establishes the root app for the new region, GitOps takes over completely. From that point forward, any new clusters added to that region are registered automatically via Git, and all workloads flow down through the ApplicationSets. (It's the infrastructure equivalent of planting a seed by hand so you can harvest the whole field automatically. Or more accurately, installing the robot that builds the robots.)

```
# root-application-set/templates/secrets.yaml
{{- range .Values.clusters }}
---
apiVersion: v1
kind: Secret
metadata:
  name: {{ .destinationName }}
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: cluster
stringData:
  name: {{ .destinationName }}
  server: {{ .server }}
  project: {{ .project }}
{{- end }}
```

The Git Generator: Auto-Discovering Services

The ApplicationSet for internal services uses ArgoCD's Git Generator to scan the helm-charts repo:

```
generators:
- git:
  repoURL: https://github.com/your-org/helm-charts.git
  revision: main
  files:
    - path: my-*/envs/prod-eu-values.yaml
```

This glob (`my-*/envs/prod-eu-values.yaml`) matches every service that has a `prod-eu-values.yaml` file. Each match generates an ArgoCD Application. **Adding a new service to production is literally creating a values file.**

The List + Multi-Source Pattern: External Charts with Custom Values

For third-party charts (Karpenter, KEDA, Sealed Secrets), we use ArgoCD's **multi-source** feature. This lets you pull the chart from one source (e.g., an OCI registry) and the values from another (your GitOps repo):

```
# When a remoteChart has a valuesFile, use multi-source
sources:
- repoURL: oci://public.ecr.aws/karpenter/karpenter
  chart: karpenter
  targetRevision: "1.8.1"
  helm:
    releaseName: karpenter
    valueFiles:
      - $values/tools/karpenter/envs/prod-us/values.yaml
- repoURL: https://github.com/your-org/gitops.git
  targetRevision: main
  ref: values    # ← This is the $values reference
```

For simpler charts, inline values work fine: (And yes, we use Sealed Secrets because base64 is an encoding, not an encryption algorithm. We're looking at you, junior devs.)


```
# When a remoteChart has inline values, use single source
source:
  repoURL: https://bitnami-labs.github.io/sealed-secrets
  chart: sealed-secrets
  targetRevision: "2.18.0"
helm:
  releaseName: sealed-secrets
  valuesObject:
    # inline values here
```

The Magic: What It Looks Like to Onboard a New Environment

Let's say the business needs a new region: **Asia-Pacific (prod-ap)**. Here's exactly what we do:

Step 1: Provision the Infrastructure (Terraform — a few minutes)

```
# Create the tfvars directory
mkdir -p tfvars/app-clusters/prod-ap

# Create the 5 config files (copy from an existing env, modify values)
cp tfvars/app-clusters/prod-eu/*.tfvars tfvars/app-clusters/prod-ap/

# Edit the files: change cluster name, region, subnet IDs, etc.
vim tfvars/app-clusters/prod-ap/common.tfvars
vim tfvars/app-clusters/prod-ap/eks-cluster.tfvars
# ... etc

# Create backend config
echo 'bucket = "my-terraform-state"
key      = "eks/prod-ap/terraform.tfstate"
region = "ap-southeast-1"' > backends/app-backends/prod-ap.hcl

# Provision!
./tf.sh prod-ap apply

# (Now go grab a coffee. EKS cluster creation still takes some time.
# We automated the config, we didn't invent time travel.)
```

Step 2: Register the Cluster in ArgoCD (GitOps — a minute or two)

```
# gitops/root-application-set/environments/prod-ap/values.yaml
global:
  repoURL: https://github.com/your-org/gitops.git
  revision: main
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
  appname: prod-ap-argo-app

clusters:
  - destinationName: prod-ap-cluster
    server: arn:aws:eks:ap-southeast-1:123456789012:cluster/my-prod-ap-cluster
    project: default

applicationSets:
  - name: prod-ap-app-set
    env: prod-ap
    path: application-sets
    valueFile: environments/prod-ap/values.yaml
    project: default
    destinationName: prod-ap-cluster
    namespace: argocd
```

Step 3: Define What Runs There (GitOps — a few minutes)

```
# gitops/application-sets/environments/prod-ap/values.yaml
global:
  repoURL: https://github.com/your-org/helm-charts.git
  revision: main
  syncPolicy:
    automated: { prune: true, selfHeal: true }

clusters:
- name: prod-ap-cluster-sets
  destinationName: prod-ap-cluster
  namespace: production
  sets:
    - name: prod-ap-app-set
      env: prod-ap-app
      path: my-*/envs/prod-ap-values.yaml
      valueFile: envs/prod-ap-values.yaml
      project: default
  remoteCharts:
    - name: karpenter
      repoURL: oci://public.ecr.aws/karpenter/karpenter
      chart: karpenter
      version: "1.8.1"
      namespace: karpenter
      values:
        replicas: 1
      settings:
        clusterName: my-prod-ap-cluster
```

Step 4: Add Service Values (Helm Charts — a few minutes per service)

```
# For each service, create a prod-ap values file
cp helm-charts/my-api-server/envs/prod-eu-values.yaml \
  helm-charts/my-api-server/envs/prod-ap-values.yaml

# Edit region-specific values (DB host, Redis endpoint, etc.)
vim helm-charts/my-api-server/envs/prod-ap-values.yaml
```

Push all three repos. ArgoCD syncs everything automatically.

It's like having an obsessive-compulsive robot constantly making sure reality matches Git. If someone decides to be a cowboy and `kubectl edit` a deployment on a Friday evening, ArgoCD's `selfHeal` simply says "No" and reverts it back in 3 seconds flat.

That's it. A new production region in a fraction of the time it used to take, with zero manual `kubectl` commands.

Principles and Practices we Followed

1. GitOps: Git as the Single Source of Truth

Every piece of configuration — infrastructure, application packaging, deployment targeting — lives in Git. There is no "we made a change in the console" or "we ran a `kubectl` command." If it's not in Git, it doesn't exist. (We like our infrastructure like we like our relationships: fully committed.)

2. Separation of Concerns

- **Terraform** owns infrastructure (clusters, IAM, networking)
- **Helm** owns application packaging (how to deploy)
- **ArgoCD** owns deployment orchestration (what goes where)

No tool reaches into another's territory. (Think of it like a healthy microservices architecture — except for the infrastructure itself. Very meta.)

3. Convention Over Configuration

The glob pattern `my-*/envs/<env>-values.yaml` is a convention. Any Helm chart following this naming is automatically discovered and deployed. Zero configuration required for new services.

4. Environment Parity Through Parameterization

The same Terraform code, same Helm charts, and same ArgoCD templates are used across all environments. The *only* difference is the values files. This eliminates "it works in staging but not in prod" drift.

5. Least Privilege with Pod Identity

Every workload gets exactly the AWS permissions it needs via Pod Identity — not a shared node role. The blast radius of a compromised pod is limited to its own permissions.

6. Blast Radius Isolation

- Tools clusters are separate from app clusters in production
- Each service is an independent ArgoCD Application (no umbrella charts)
- State files are per-environment (no shared Terraform state)

7. Self-Healing and Drift Detection

ArgoCD's `selfHeal: true` means if someone manually changes something in the cluster, ArgoCD reverts it. The Git repo always wins.

8. Declarative Add-on Management

Add-ons are categorized by their lifecycle requirements:

- **Bootstrap addons** (vpc-cni, kube-proxy): installed with the cluster, before nodes
- **Post-node addons** (CoreDNS, metrics-server): installed after nodes are ready
- **IAM-sensitive addons** (EBS CSI, EFS CSI): installed with Pod Identity associations

What This Architecture Enables

Capability	How It Works
New region in minutes	Copy tfvars + add values files → <code>./tf.sh apply</code> → push GitOps config
Zero config drift	Same code across all environments; only values files differ
Self-healing clusters	ArgoCD <code>selfHeal: true</code> reverts any manual cluster changes automatically
Fully automated deployments	Push to Git → ArgoCD detects and syncs — zero manual intervention

Capability	How It Works
Complete deployment visibility	ArgoCD dashboard is the single source of truth for what runs where
Intelligent node scaling	Karpenter provisions right-sized nodes in seconds via EC2 Fleet API
Per-service least-privilege	EKS Pod Identity gives each workload only the AWS permissions it needs
Developer self-service	Follow the naming convention, create a values file, push — no DevOps ticket needed
Management plane isolation	Separate tools cluster ensures ArgoCD survives app cluster failures
Safe multi-team collaboration	Per-environment S3 state + DynamoDB locking prevents Terraform conflicts
Compliance-ready data residency	Region-isolated clusters with parameterized configs per geography

Sample Code

We've published a complete, minimal sample implementation that follows all the patterns described in this framework. You can find it in the `sample-code/` directory alongside this article, covering:

- **Terraform:** Full EKS provisioning with IAM module, Karpenter, ArgoCD capability, and multi-env tfvars
- **GitOps:** Root ApplicationSet + leaf ApplicationSet templates with both Git and List generators
- **Helm Charts:** A sample API server chart and Karpenter CRD config chart

Final Thoughts

Building a framework like this is an investment. It takes deliberate design, careful dependency ordering, and a willingness to think in layers. But the payoff is a platform that scales by configuration,

not by engineering effort.

The return on that investment is immediate: when the business says "we need production in Asia by next quarter" or a new compliance requirement demands workload isolation in a specific geography, the answer is a directory of config files and a `git push`. No war room. No infrastructure sprint. No surprises. And when you need to add AWS resources beyond EKS — databases, caches, queues, storage — the same modular, parameterized pattern applies. Add a module, add a `tfvars` entry, and the framework handles the rest.

The best platform is one that makes the right thing the easy thing. Adding a region should be boring. Adding a microservice should be a values file. Recovering from drift should be automatic.

Design for modularity. Build for consistency. Then let the platform do its job. (And maybe go home before midnight for once.)